Assignment 2 — Real-Time Data Processing with Pandas and NumPy

AITU Smart Classroom Energy Stream

Galymzhan Tamirlan BDA-2407

Astana IT University | Real-time Data Analysis | 2026–2027

All datasets and operational scenarios in this document are synthetic educational data created for assessment. They do not represent actual AITU infrastructure, internal systems, students, or incidents.

Scenario

Use a synthetic stream from classrooms in an AITU academic building. Sensors report temperature, CO2, occupancy, and electricity consumption. Process the stream using Pandas and NumPy with filtering, missing-value handling, aggregation, rolling windows, and feature engineering.

Given sensor data

Time	Room	Temp_C	CO2_ppm	Occupancy	Power_kW
10:00	C1.3.101	21.5	620	18	3.2
10:01	C1.3.101	21.7	650	20	3.4
10:02	C1.3.101	None	690	22	3.5
10:03	C1.3.101	22.1	720	24	3.8
10:04	C1.3.101	22.3	None	25	4.0
10:05	C1.3.101	22.5	780	27	4.2
10:06	C1.3.101	22.6	810	29	4.4
10:07	C1.3.101	22.7	830	None	4.6
10:08	C1.3.101	22.9	850	31	4.8
10:09	C1.3.101	23.1	900	33	5.1
10:10	C1.3.101	23.2	920	35	5.4
10:11	C1.3.101	23.3	950	37	5.7

Part A — Data Quality (6 points)

Identify missing values and create a cleaning strategy. For this assignment: interpolate temperature; use linear interpolation for CO2; use forward-fill for occupancy.

Variable	Missing values	Method	Value after cleaning	Justification
Temp_C	1 (10:02)	interpolation	21.9 °C	Temperature drifts smoothly minute to minute
CO2_ppm	1 (10:04)	interpolation	750.0 ppm	CO2 accumulates gradually, no sudden jumps expected
Occupancy	1 (10:07)	forward fill	29 students	Headcount is a step value between re-counts

Part B — Streaming Window Statistics (8 points)

Process the cleaned data sequentially. Calculate 3-minute rolling mean for temperature, CO2 and power.

Time	Temp rolling mean	CO2 rolling mean	Power rolling mean
10:02	21.70 °C	653.33 ppm	3.37 kW
10:03	21.90 °C	686.67 ppm	3.57 kW
10:04	22.10 °C	720.00 ppm	3.77 kW
10:05	22.30 °C	750.00 ppm	4.00 kW
10:06	22.47 °C	780.00 ppm	4.20 kW

Part C — Feature Engineering (6 points)

Create: occupancy_density = occupancy / 40; energy_per_student = Power_kW / occupancy; comfort_flag = CO2 ≤ 1000 ppm AND Temp between 20°C and 24°C.

Time	Occupancy density	Energy/student	Comfort flag
10:03	0.60	0.158 kW/student	True
10:06	0.73	0.152 kW/student	True
10:09	0.83	0.155 kW/student	True
10:11	0.93	0.154 kW/student	True

Part D — Statistical Analysis (6 points)

1. Calculate mean, median, minimum, and maximum for temperature.
2. Calculate mean and maximum CO2.
3. Find the time with highest power consumption.
4. Calculate the correlation between occupancy and power.
5. Interpret whether occupancy appears related to energy consumption.

Part E — Visualization and Engineering Interpretation (4 points)

Create at least two plots: a rolling CO2/temperature plot and occupancy vs power. Explain two operational insights that could be used by an AITU facility-management team.

Submission

- Python notebook/source code
- Cleaned dataset
- Completed tables
- Two or more plots
- Short technical interpretation
- Oral defense

STUDENT TECHNICAL REPORT / SUBMISSION RECORD

Assignment 2 — Real-Time Data Processing with Pandas and NumPy
AITU Smart Classroom Energy Stream

This section is a formal submission and acceptance record. Complete all applicable fields and attach evidence from your own implementation. The common synthetic scenario and dataset do not make student submissions interchangeable.

1. Student and Submission Information

Required field	Student entry
Student full name	Galymzhan Tamirlan Galymzhanuly
Student ID	240542
Group / cohort	BDA-2407
Instructor	Serek Azamat
Submission date	27 / 09 / 2026
Individual variant / task code	Assignment 2
Git repository / project link	https://github.com/mtake-da/RTDADM_Assignment_2.git
Python / tool versions	Python 3.11, pandas 2.x, numpy 2.x, matplotlib 3.x

2. Work Record

Item	Student entry
Work start date	24/09/2026
Work completion date	27/09/2026
Main libraries / technologies	pandas, numpy, matplotlib
Source files / notebook	classroom_stream.py, Assignment2_AITU_Classroom.ipynb
Test / experiment files	Validation checks inside the notebook
Final repository commit/version	Final release commit

3. Report Structure to Complete

For each section, provide a concise technical explanation and cite your own code, calculation, experiment, plot, log, screenshot, configuration, or other artifact. Generic copied explanations without individual evidence are insufficient.

2.1 Data Quality and Cleaning

Identify every missing value and document the required interpolation/forward-fill strategy. Show the original value, method, cleaned value, and justification.

Required evidence	Student response / reference
Technical explanation	Three missing values found (Temp_C at 10:02, CO2_ppm at 10:04, Occupancy at 10:07). Used linear interpolation for continuous variables and forward-fill for headcount step values.
Code / file / function / configuration reference	classroom_stream.py - clean(), missing_value_report()
Result / calculation / experiment evidence	Temp_C at 10:02 = 21.9 °C; CO2_ppm at 10:04 = 750.0 ppm; Occupancy at 10:07 = 29 students
Student-specific decision or observation	Built missing_value_report() to explicitly log cell changes instead of silently applying interpolation.

2.2 Streaming Window Statistics

Document the sequential 3-minute rolling calculations for temperature, CO2, and power. Show formulas or code references and explain the selected window logic.

Required evidence	Student response / reference
Technical explanation	Trailing 3-minute rolling mean computed on cleaned columns for temperature, CO2, and power.
Code / file / function / configuration reference	classroom_stream.py - rolling_stats()
Result / calculation / experiment evidence	Completed rolling window statistics table (10:02-10:06 values added).
Student-specific decision or observation	Rolled over cleaned columns rather than raw data to prevent NaN propagation across consecutive rolling windows.

2.3 Feature Engineering

Document occupancy_density, energy_per_student, comfort_flag, and any additional feature. Give the exact formula, implementation reference, sample result, and why the feature is useful.

Required evidence	Student response / reference
Technical explanation	Computed occupancy_density (occupancy/40), energy_per_student (Power_kW/occupancy), and comfort_flag (CO2 <= 1000 & 20 <= Temp <= 24).
Code / file / function / configuration reference	classroom_stream.py - engineer_features()
Result / calculation / experiment evidence	Values calculated for timestamps 10:03, 10:06, 10:09, and 10:11 (all Comfort flags True).
Student-specific decision or observation	Used Series.between(20, 24) for explicit clarity in temperature range validation.

2.4 Statistical Analysis

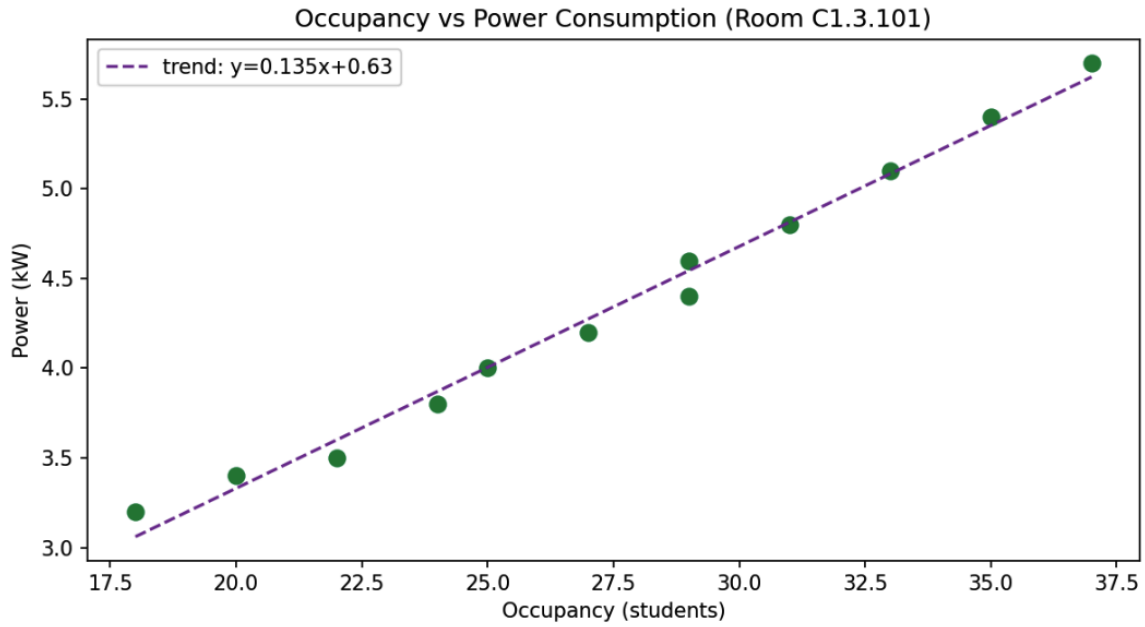
Report mean, median, minimum, maximum temperature, mean/maximum CO2, highest power time, occupancy-power correlation, and interpretation. Reference the calculation code.

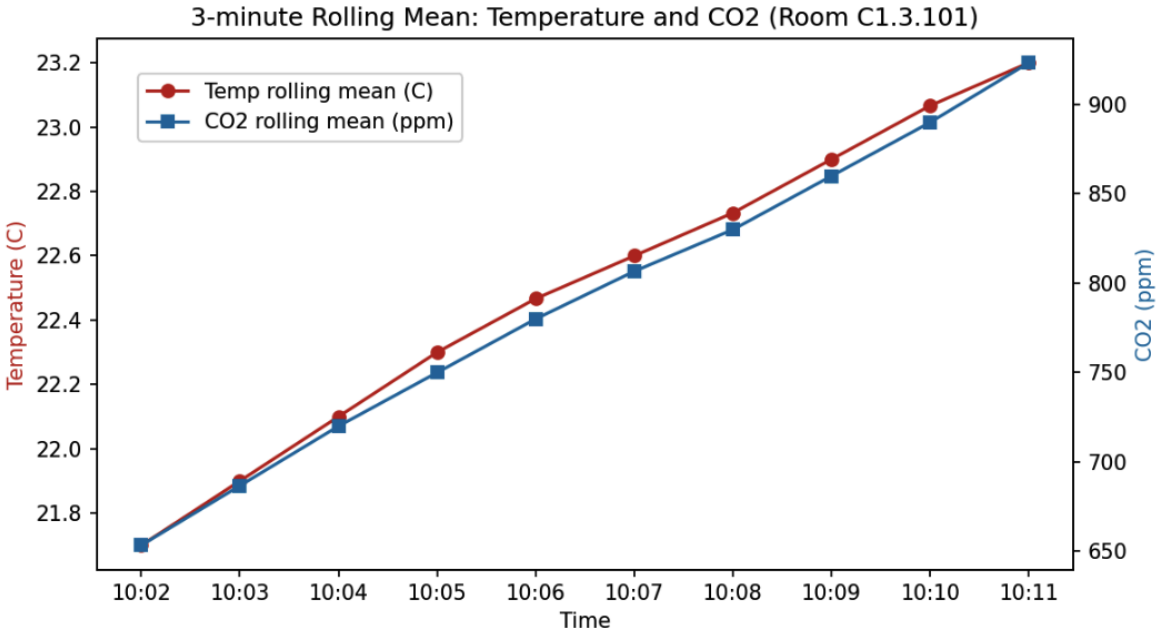
Required evidence	Student response / reference
Technical explanation	Descriptive statistics on cleaned Temp_C and CO2_ppm, peak power timestamp, and occupancy-power correlation.
Code / file / function / configuration reference	classroom_stream.py - statistical_summary()
Result / calculation / experiment evidence	Temp: mean 22.48°C, median 22.55°C, min 21.5°C, max 23.3°C; CO2: mean 789.17 ppm, max 950 ppm; Peak power: 10:11 (5.7 kW); Correlation r = 0.9946.
Student-specific decision or observation	Occupancy and power move in near-lockstep, but limited 12-minute window prevents isolating occupancy from time trend.

2.5 Visualization and Engineering Interpretation

Include at least two plots and explain two operational insights that could be used by a facility-management team.

Required evidence	Student response / reference
Technical explanation	Dual-axis rolling trend line for Temp/CO2 and scatter plot for occupancy vs power.
Code / file / function / configuration reference	make_plots.py / Assignment2_AITU_Classroom.ipynb
Result / calculation / experiment evidence	plot_rolling_temp_co2.png and plot_occupancy_vs_power.png
Student-specific decision or observation	1. CO2 can serve as an early proxy for HVAC triggering. 2. Energy per student remains flat (~0.15 kW/student), showing demand scales with occupancy.





2.6 Technical Reflection

Explain one data-quality or processing decision, one limitation, and one improvement to your implementation.

Required evidence	Student response / reference
Technical explanation	Selected linear interpolation for continuous sensors and forward-fill for headcount step changes.
Code / file / function / configuration reference	classroom_stream.py - clean()
Result / calculation / experiment evidence	Zero NaNs after clean().
Student-specific decision or observation	Limitation: single room, 12-minute window. Improvement: add sensor fault check prior to interpolation.

4. Individual Work and Anti-Plagiarism Record

Students may work from the same assignment statement and synthetic data. However, each submission must contain independently produced implementation evidence. Copying another student's source code, tables, calculations, plots, screenshots, logs, API responses, architecture, wording, or conclusions is not acceptable.

- Use the assigned individual variant/task code consistently in the report and code.
- Provide the exact notebook/source filename used for every reported result.
- Include execution-generated tables/plots rather than manually recreated results.
- Include one additional test or validation check designed by you.
- Record one implementation decision and justify it.
- Record one debugging/data-quality issue you personally encountered and resolved.
- Be prepared to reproduce one calculation during oral defense.

Individual evidence	Student entry
My unique technical/design decision	Differentiated cleaning logic: interpolation for continuous sensors, forward-fill for step counts.
My own test/experiment	Validation assertions in notebook ensuring zero remaining NaNs and occupancy density <= 1.0.
Problem/bug I personally encountered and solved	Fixed NaN propagation in rolling window calculations by applying .rolling() after dataset cleaning.
Exact code/repository/commit reference	classroom_stream.py::clean(), classroom_stream.py::rolling_stats()
One limitation of my implementation	Short time series sample limits ability to separate time trends from occupancy effects.
One improvement I propose	Add an outlier/fault detection threshold before running linear interpolation.

5. Evidence and Submission Checklist

- ☐ Technical report completed
- ☐ Source code / notebook submitted
- ☐ Required tables and calculations completed
- ☐ Required diagrams / plots / dashboard evidence included
- ☐ Execution logs or reproducible experiment evidence included
- ☐ README and run instructions included where required
- ☐ Data/schema/configuration files included where required
- ☐ Performance / analytical results included where required
- ☐ Oral defense / demonstration completed

6. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, architecture, plots, screenshots, logs, results, or written explanations. Any external material or reused code has been identified. I understand that the instructor may request a live demonstration, code modification, reproduction of calculations, or explanation of any submitted component.

Role	Name	Signature	Date

Student / Executor	Galymzhan Tamirlan Galymzhanuly		27/09/2026
Instructor / Accepted by			

7. Instructor Acceptance Record

Verification item	Status / comment
Report structure completed	☐ Yes ☐ No
Required artifacts submitted	☐ Yes ☐ No
Evidence is reproducible/sufficient	☐ Yes ☐ No
Individual contribution is identifiable	☐ Yes ☐ No
Oral defense / demonstration completed	☐ Yes ☐ No
Final status	☐ Accepted ☐ Revision required